MissLearn User Guide

Amanda S. Barnard

August 2026

Contents

MissLearn User Guide

Version 0.9.2 · Full-information maximum likelihood estimators with native missing-data support, following the scikit-learn estimator API.

MissLearn fits models directly on matrices containing `NaN`. Nothing is deleted, nothing is filled in. Every estimator maximises the likelihood of the data you actually observed, marginalising analytically over the entries you did not.

Contents

---

1. Getting started

1.1 Installation

```
pip install misslearn
```

Requires Python 3.9 or later, `numpy>=1.22`, `scipy>=1.8` and `scikit-learn>=1.1`. Optional extras: `pandas` for DataFrame support, `matplotlib` for the plotting helpers, `trees` for the NaN-native gradient-boosted learners that `MissEnsemble` accepts as members, and `all` for everything.

```
pip install "misslearn[all]"
```

1.2 A first model

```
import numpy as np
from MissLearn import MissLinear

rng = np.random.default_rng(0)
X = rng.normal(size=(500, 6))
y = X @ rng.normal(size=6) + rng.normal(scale=0.5, size=500)
X[rng.random(X.shape) < 0.2] = np.nan      # 20% of cells missing
```

```
model = MissLinear().fit(X, y)            # no imputation, no deletion
print(model.coef_)
print(model.score(X, y))
model.summary()                           # coefficients, SEs, p-values, AIC
```

There is no imputation step and no `dropna`. The `NaN` entries are part of the likelihood, not an obstacle to it.

1.3 The estimator API

MissLearn estimators are scikit-learn estimators. They subclass `BaseEstimator` together with `RegressorMixin` or `ClassifierMixin`, so the conventions you already know all hold:

Convention	Behaviour
`fit(X, y)`	Returns `self`. Accepts `NaN` in `X`, and in `y` for the likelihood models.
`predict(X)`	Accepts `NaN`. Missing features are marginalised, not filled.
`predict_proba(X)`	Classifiers. Integrates over the missing features rather than plugging in a point estimate.
`score(X, y)`	R^2 for regressors, accuracy for classifiers.
`get_params()` / `set_params()`	Full support, so `clone`, `GridSearchCV` and `Pipeline` all work.
Trailing underscore	`coef_`, `classes_`, `n_features_in_` and so on exist only after `fit`.
`n_features_in_`, `feature_names_in_`	Set at fit time. Names come from a DataFrame's columns when you pass one.

Every family ships three names: a regressor, a classifier, and a dispatcher that picks between them by inspecting `y`.

```
from MissLearn import MissRidgeRegressor, MissRidgeClassifier, MissRidge

MissRidge().fit(X, y_continuous)   # dispatches to MissRidgeRegressor
MissRidge().fit(X, y_binary)       # dispatches to MissRidgeClassifier
```

Use the explicit name when you know the task; the dispatcher is a convenience, and being explicit is clearer in code others will read.

Absent outcomes. `NaN` in `y` is supported by the likelihood families and is not the same thing as a row being useless. A row whose outcome was never recorded still carries information about the joint distribution of the predictors, and full-information estimation uses it for exactly that: the row informs `mu_X` and `Sigma_X` while contributing no outcome term. This is one of the differences from listwise deletion that the method is for, so a target that is half absent is a normal input rather than a degenerate one.

The exception is a target with *nothing* observed, which every estimator refuses:

```
MissLinear().fit(X, np.full(len(X), np.nan))
# ValueError: MissLinear: y is entirely NaN, so there is no observed outcome
# to fit. Absent entries in y are supported and still inform the feature
# distribution, but at least one outcome must be observed.
```

With no observed outcome there is no supervised problem left to solve. Before this release the three families disagreed about that: one refused, one fitted and then predicted `NaN` from `NaN` coefficients, and one fitted a zero model and returned finite numbers from no data at all. The refusal now lives in the shared input validation, so every estimator gives the same answer.

---

2. Statistical background

2.1 Missingness mechanisms

The right treatment depends on why the values are absent. The standard taxonomy is due to Rubin.

MCAR, missing completely at random. The probability of being missing is independent of everything, observed or not. Listwise deletion is unbiased here but still throws away information.

MAR, missing at random. The probability of being missing depends only on values you *did* observe. A lab test ordered whenever a recorded symptom is present is MAR. This is the assumption under which FIML is consistent and efficient, and it is the assumption MissLearn's estimators make.

MNAR, missing not at random. The probability depends on the unobserved value itself. Incomes missing because they are high, or symptoms unrecorded because the patient was too unwell to attend. No method recovers the truth from the data alone. What you can do is quantify how far a conclusion would have to be pushed before it changes, which is what `MissSensitivity` is for (section 8.3).

MCAR is testable, and `MissDiagnostic.little_mcar_test` tests it. MAR is not testable against MNAR from the data. That asymmetry is unavoidable and worth stating plainly rather than hiding behind a diagnostic.

2.2 Why deletion and imputation fall short

Listwise deletion discards every row with any missing entry. With p features each independently missing at rate r, the expected fraction of rows surviving is (1 - r)^p, so at 20% missing over ten features roughly one row in ten remains. Beyond the loss of power, the survivors are a biased sample under anything but MCAR. In the worked Parkinson's example this is not a subtle effect: deletion diverges to an R^2 of about -10,300, because the surviving design is near-singular.

Single imputation fills each hole with one value and then proceeds as if the value had been measured. Downstream standard errors are too small, because nothing in the pipeline records that the value was invented.

Multiple imputation does account for that uncertainty, by drawing *m* completed data sets and pooling by Rubin's rules. It is a sound method, and MissLearn provides it (`MissImputer`, section

5). Its cost is that you fit *m* models, tune an imputation model as well as an analysis model, and inherit any mismatch between the two.

Full-information maximum likelihood takes a different route. It writes the likelihood of what was observed and maximises it directly. Under MAR the estimates are consistent and asymptotically efficient, which is the guarantee multiple imputation approximates, obtained from one deterministic fit.

2.3 Full-information maximum likelihood

For a row with observed entries indexed by o, the contribution to the log-likelihood uses only the marginal distribution of those entries:

```
log L = sum_i log p(x_i,o ; theta)
```

For a joint Gaussian working model with parameters (mu, Sigma), the marginal of any subset is Gaussian with the corresponding sub-vector and sub-matrix, so each row's contribution is available in closed form. There is no integral to approximate and no value to invent.

Rows sharing the same missingness pattern share the same sub-matrix, so the implementation groups rows by pattern and factorises once per pattern rather than once per row. Cost is therefore $O(G \cdot p^3)$ where G is the number of *distinct* patterns, not $O(n \cdot p^3)$. This matters in practice: blockwise missingness, where whole groups of columns go missing together, keeps G small and is dramatically cheaper than scattered missingness at the same rate.

2.4 The joint-Gaussian working model and its limits

Most MissLearn estimators marginalise over a joint Gaussian working model for the features. Being explicit about that assumption is important, because it is where the method is strongest and where it fails.

It is well matched to continuous, roughly elliptical features. It is strained by heavy tails, which the copula transform (section 4.3) can absorb, and by binary indicator columns, where a Gaussian density over a set of zero/one columns is badly misspecified. The generative family (`MissBayes`) is the most exposed, since it models the feature density directly.

The working model concerns the *features*. The response model is separate: a linear conditional mean for `MissLinear`, a logistic link for `MissLogistic`, a kernel for `MissSupport` and `MissGaussian`. When a model underperforms it is worth asking which of the two assumptions is the binding one. In the shipped thyroid example the answer turns out to be the response model, not the Gaussian working model, and switching family recovers the gap.

---

3. Supervised learning

Each subsection gives the formulation, a usage sketch, practical guidance, and the attributes the fit exposes.

3.1 Linear models

`MissLinear`, `MissLogistic`

The workhorses. Both estimate a joint model for the features together with a conditional model for the response.

```
MissLinear(max_iter=2000, tol=1e-07, method='L-BFGS-B',
           compute_se=True, copula='auto', warm_start=False)

MissLogistic(max_iter=2000, tol=1e-07, method='L-BFGS-B', n_quadrature=20,
             compute_se=True, l2_reg=0.01, copula='auto', warm_start=False)
```

Formulation. `MissLinear` models [y, X] as jointly Gaussian and reads the regression of y on X from the fitted moments. `MissLogistic` keeps a Gaussian model for X and a logistic link for y, and integrates the sigmoid over the conditional distribution of the missing features by Gauss-Hermite quadrature with `n_quadrature` nodes.

Estimation. Both fit in two stages. The joint moments are obtained by EM, which reaches the FIML maximum far more cheaply than quasi-Newton search, and a short L-BFGS-B pass then polishes the result. Convergence uses two criteria: `ftol = tol` on the relative function change and `gtol = tol * 1e-2` on the gradient norm, so the gradient test is a hundred times tighter.

Usage.

```
from MissLearn import MissLogistic
clf = MissLogistic().fit(X, y)
clf.summary()                       # coefficients, SEs, z, p, AIC, BIC
clf.conf_int(alpha=0.05)
proba = clf.predict_proba(X_new)    # integrates over missing features
```

Tips on practical use.

- Set `compute_se=False` when you only want predictions. Standard errors cost a numerical Hessian and are pure overhead if you never read them.
- `predict_proba` and `sigmoid(decision_function(x))` do not agree on partial rows, and should not. `predict_proba` integrates the sigmoid over the conditional distribution of the missing features, which pulls probabilities towards 0.5 as uncertainty grows. `decision_function` plugs in conditional means. The first is the honest answer.
- `warm_start=True` reuses the previous fit's parameters when refitting the same estimator on related data. It is deliberately *not* used across cross-validation folds; see section 6.1.
- `l2_reg` on `MissLogistic` defaults to a small non-zero value because the likelihood is unbounded under perfect separation.

Attributes. `coef_`, `intercept_`, `coef_std_`, `loglik_`, `aic_`, `bic_`, `converged_`, `mu_joint_` and `Sigma_joint_` (`MissLinear`) or `mu_X_` and `Sigma_X_` (`MissLogistic`), plus `classes_` for the classifier.

3.2 Penalized linear models

`MissRidgeRegressor`, `MissRidgeClassifier`, `MissRidge`, `MissLASSORegressor`, `MissLASSOClassifier`, `MissLASSO`

```
MissRidgeRegressor(alpha=1.0, max_iter=2000, tol=1e-07, method='L-BFGS-B',
                   compute_se=True, copula='auto')

MissLASSORegressor(alpha=1.0, max_iter=3000, tol=1e-07, method='L-BFGS-B',
                   compute_se=False, copula='auto')
```

Formulation. The same FIML likelihood with an L2 or L1 penalty on the regression coefficients. The penalty applies to the response model only, not to the feature moments, so it shrinks the estimates you interpret without distorting the marginalisation.

Tips on practical use.

- `alpha` needs tuning, and it needs tuning by cross-validation rather than by eye. When comparing against a conventional pipeline, tune the penalty on *both* sides over the same grid. A likelihood that marginalises and a pipeline that imputes have different effective sample sizes, so they do not want the same amount of regularisation, and fixing one value for both quietly favours whichever it happens to suit.
- LASSO defaults to `compute_se=False` and `max_iter=3000`. Standard errors for L1-penalised estimates are not straightforward to interpret, and the non-smooth objective needs more iterations.
- Neither penalized family exposes `warm_start`; that parameter belongs to `MissLinear` and `MissLogistic` only.

3.3 Generative models

`MissBayesRegressor`, `MissBayesClassifier`, `MissBayes`

```
MissBayesRegressor(var_smoothing=1e-09, copula='auto',
                   structure='full', shrinkage='auto')
```

Formulation. A full-covariance Gaussian model per class (classification) or a joint Gaussian over [y, X] (regression), with prediction by Bayes' rule. Because the model *is* a density over the features, marginalising a missing entry is exact and immediate, with no optimisation at all. Fits are close to instantaneous.

Tips on practical use.

- This is the family that gains most from marginalisation, precisely because it depends on the feature density itself. On the shipped air-quality example it is the one family that pulls clearly ahead.
- It is also the family most exposed to misspecification. A full-covariance Gaussian over indicator columns is badly wrong, and the symptom is confident, poorly calibrated probabilities: on the thyroid data it reaches a Brier score of 0.60 while the discriminative families sit near 0.03. Check the Brier score, not just accuracy or AUC.
- `structure='full'` estimates a full covariance per class. With many features and few rows per class, use the shrinkage control rather than fighting a singular matrix.

3.4 Nearest neighbours

`MissNeighborsRegressor`, `MissNeighborsClassifier`, `MissNeighbors`

```
MissNeighborsRegressor(n_neighbors=5, weights='distance',
                       metric='euclidean', copula='auto')
```

Formulation. Distances are not computed on imputed values. The estimator computes the *expected* squared distance between two rows under the fitted joint Gaussian, which adds the conditional variance of the unobserved coordinates rather than pretending they equal their means. Two rows that are both missing a coordinate are correctly treated as less certainly close than two rows that agree on it.

Tips on practical use.

- Neighbour methods degrade with dimension whether or not data are missing. Missingness compounds this, because every unobserved coordinate contributes variance to every distance.
- `weights='distance'` is the default and is usually right here: it downweights neighbours whose distance is uncertain.
- A nearest-neighbour probe is a poor proxy for what a *kernel* can find. If `MissNeighbors` does not help, do not conclude that non-linearity is absent; try `MissSupport` before deciding.

3.5 Support vector machines

`MissSupportRegressor`, `MissSupportClassifier`, `MissSupport`

```
MissSupportRegressor(C=1.0, epsilon=0.1, kernel='rbf', gamma='scale',
                     degree=3, coef0=0.0, copula='auto')

MissSupportClassifier(C=1.0, kernel='rbf', gamma='scale',
                      degree=3, coef0=0.0, copula='auto')
```

Formulation. The kernel matrix is replaced by its expectation under the fitted joint Gaussian. For the RBF kernel this expectation is available in closed form, and the result remains positive semi-definite, so the usual convex machinery applies unchanged.

Tips on practical use.

- This is the family to reach for when the response is non-linear in the features. In the thyroid example it lifts ROC-AUC from 0.934 to 0.975 over the linear family on identical folds.
- `C` matters and interacts with the missing rate. Tune it, and tune the conventional comparator over the same grid.
- Cost grows quickly with n. Expect this to be one of the slower families.

3.6 Gaussian processes

`MissGaussianRegressor`, `MissGaussianClassifier`, `MissGaussian`

```
MissGaussianRegressor(kernel='rbf', ard=False, n_restarts=3,
                      noise_var_init=0.1, copula='auto')

MissGaussianClassifier(kernel='rbf', ard=False, n_restarts=3,
                       max_iter_newton=50, copula='auto')
```

Formulation. Exact GP inference with the kernel marginalised over the missing entries, giving calibrated Bayesian predictive intervals that widen appropriately for incomplete query rows. The classifier uses the Laplace approximation with a Newton iteration.

Tips on practical use.

- Exact inference is $O(n^3)$. Keep n below roughly 1,000 to 2,000. `MissRecommender` vetoes this family above `gp_max_n`, default 1,000.
- **The classifier costs about nine times the regressor.** Measured at n = 90, p = 3 on an idle machine, one fit takes roughly **1.2 s** for `MissGaussianRegressor` and **11 s** for `MissGaussianClassifier`. Under scikit-learn's 54-check estimator battery, which refits many times over, the same pair take 94 s and 2,723 s. The difference is the Laplace approximation: the classifier finds the posterior mode with up to `max_iter_newton` Newton steps inside *every* objective evaluation, and the optimiser repeats that for each of `1 + n_restarts` restarts. Four nested loops around a cubic kernel is why a few thousand rows stops looking slow and starts looking like a hang.
- **What to turn down, in rough order of what it buys.** `n_restarts` (default 3, so four optimiser runs) is the cheapest thing to lower and usually costs little on a well-conditioned problem. Then subsample the rows, since the cost is cubic in exactly that. Then `max_iter_newton` on the classifier. If none of that is enough, `MissSupport` and `MissNeighbors` cover much of the same ground with a kernel or a distance rather than a full posterior.
- The regressor centres the response internally but does not rescale it. The prior has mean zero, so an uncentred response would revert towards 0 rather than towards the response mean away from the training data.
- `ard=True` gives a length scale per feature. Informative when features differ in relevance, and more expensive to fit.
- If you compare against a conventional GP, give the comparator a learnable noise term. A GP confined to near-exact interpolation diverges on incomplete data, because imputed rows become near-duplicate inputs with different targets. That is a misconfigured baseline, not a win.

3.7 Linear mixed-effects models

`MissMixedRegressor`, `MissMixedClassifier`, `MissMixed`

```
MissMixedRegressor(max_iter=2000, tol=1e-07, method='L-BFGS-B',
                   compute_se=True, copula='auto', fe_ridge=0.01)

MissMixedClassifier(n_quadrature=20, max_iter=2000, tol=1e-07,
                    method='L-BFGS-B', compute_se=True, copula='auto')
```

Formulation. A random-intercept model, $y_{ij} = \beta_0 + \beta \cdot x_{ij} + b_i + \varepsilon_{ij}$ with $b_i \sim N(0, \tau^2)$, for repeated-measures, longitudinal or clustered data. Missing features are marginalised as elsewhere. The classifier is the GLMM analogue, integrated by Gauss-Hermite quadrature.

Usage. `groups` is required and identifies the cluster of each row.

```
from MissLearn import MissMixedRegressor
m = MissMixedRegressor().fit(X, y, groups=subject_id)
y_hat = m.predict(X_new, groups=new_ids)   # known ids get their BLUP
print(np.sqrt(m.tau_sq_), m.icc_)          # between-group sd, ICC
```

Tips on practical use.

- Read `icc_` before anything else. It tells you how much of the variation is between groups rather than within them, and therefore how much of your apparent accuracy comes from having seen the group before. In the shipped Parkinson's example the ICC is 0.935, and predicting for a monitored patient gives RMSE 2.60 against 10.43 for a new one. Reporting the first number for a task that is really the second overstates accuracy by a factor of four.
- Cross-validate grouped by cluster. A shuffled split leaks group identity through the repeated measures and is answering a different question.
- `tau_sq_` is a *variance*. Take its square root for the standard deviation. There is no `tau_` attribute.
- `fe_ridge` puts a small ridge on the fixed effects and defaults to 0.01. It keeps the fixed-effect block conditioned when groups are unbalanced. Lower it only after checking the fit is stable without it.

Attributes. Adds `blup_`, `tau_sq_`, `sigma_sq_`, `icc_` and `groups_fit_`.

3.8 Ensemble methods

`MissEnsemble`

```
MissEnsemble(estimator=None, estimators=None, n_estimators=100, weights=None,
             bootstrap=True, max_samples=1.0, max_features=1.0,
             oob_score=False, n_jobs=1, random_state=None)
```

Bagging over any NaN-tolerant base learner, in two modes.

Homogeneous: one `estimator`, cloned `n_estimators` times over bootstrap resamples.

```
MissEnsemble(estimator=MissSupportClassifier(kernel='rbf', C=16),
             n_estimators=10, oob_score=True).fit(X, y)
```

Heterogeneous: an explicit `estimators=[(name, est), ...]` list, optionally weighted.

```
MissEnsemble(estimators=[('svm', MissSupportClassifier()),
                         ('logistic', MissLogistic())],
             weights=[3, 1]).fit(X, y)
```

Tips on practical use.

- Choose the model class first, on evidence, and only then decide whether to ensemble it. Bagging a well-chosen base learner is reliable; a heterogeneous ensemble is not a substitute for the choice. Averaging decorrelates errors but also averages in the members' skill, so mixing a strong model with weaker ones usually drags the strong one down whatever the weights say.
- `oob_score=True` gives free internal validation from the bootstrap out-of-bag rows, and the per-member scores are the quickest way to see whether a heterogeneous mix is carrying a passenger.
- `summary(feature_names=...)` labels the importance table with real names. Without it the rows read `X17`, which cannot be checked against a data dictionary.

- Members are validated at construction. Passing a plain `LogisticRegression` raises a descriptive `ValueError`, because it cannot be fitted on NaN. Third-party NaN-native gradient-boosted learners are accepted as members; that is a library capability, and combining them with a likelihood model in one predictor is not the same thing as benchmarking one against the other.

3.9 Multiclass strategies

```
MissMulticlass(estimator, strategy='ovr')
```

Extends any binary MissLearn classifier to multiple classes by one-vs-rest decomposition with NaN-preserving label encoding and row-normalised probabilities.

```
from MissLearn import MissMulticlass, MissLogistic
clf = MissMulticlass(MissLogistic()).fit(X, y_three_class)
```

---

4. Preprocessing and validation

4.1 prefit_check

```
prefit_check(X, y=None, model_name='', feature_names=None,
             categorical_threshold=10, missingness_threshold=0.70,
             scale_ratio_threshold=1000.0, kurtosis_threshold=7.0,
             n_gaussian_threshold=1000, raise_on_error=True,
             emit_warnings=True, copula_configured=False)
```

A standalone compatibility audit. It flags all-NaN columns, extreme missingness, constant features, wild scale ratios, heavy tails, and categorical-looking columns, raising or warning as configured.

Pass `feature_names` (or a DataFrame) so the report names the measurement rather than `X4`.

4.2 MissPreprocessor

```
MissPreprocessor(estimator, encode='auto', categorical_threshold=10,
                 drop='first', validate=True, raise_on_error=True,
                 verbose=True, feature_names=None)
```

Wraps any MissLearn model, runs `prefit_check` at fit time, and adds NaN-preserving one-hot encoding for categorical inputs. Encoding a categorical column must not turn a missing category into a row of zeros, which would be indistinguishable from an observed absence of every level; the encoder preserves the `NaN` so the likelihood can marginalise it.

Names resolve in priority order: the `feature_names` argument, then a DataFrame's columns, then an existing `feature_names_in_`, then generic `X0..Xp`.

4.3 The copula transform

Every estimator takes `copula`, which accepts `False`, `True` or `'auto'`.

The transform maps each feature's marginal to a standard normal by its ranks, fits in the transformed space, and maps back. It relaxes the marginal normality assumption while keeping the dependence structure that makes marginalisation tractable.

With `'auto'` the model decides from the data. Use it when `prefit_check` reports high kurtosis, but treat the note as a prompt to test rather than an instruction: heavy margins do not always mean the joint-normal working model is what limits the fit. Run the comparison and let it decide. `copula_used_` reports what was chosen.

Which columns it touches Not all of them, and this matters for reading coefficients.

A column with fewer than **three distinct observed values** is left on its own scale, in both directions. Sklar's theorem, which is what licenses the whole construction, identifies a copula uniquely only when the marginals are continuous. For a discrete marginal the empirical distribution function is a step, the copula is not identified, and mapping ranks to normal scores is a relabelling of the categories rather than a route to normality. One-hot dummies and binary indicators therefore pass straight through.

The same rule governs the automatic decision. `'auto'` looks only at columns it would actually transform, so an indicator no longer votes. This is not a refinement but a correction: an indicator with prevalence outside roughly `[0.25, 0.75]` has skewness above 1 *by construction*, so a matrix of dummies used to demand a transform that could not help it. On a one-hot encoded credit data set with 37 columns, 31 of them dummies, `'auto'` now transforms the six continuous financial amounts, whose skewness runs from 1.1 to 13.1, and leaves the dummies alone.

A column with nothing left to estimate from, meaning zero or one observed value, takes the same route. It used to raise, which made the copula the only stage that could refuse data the rest of the library accepts; the same matrix fitted without complaint under `copula=False`. Since `'auto'` is the default, any cross-validation fold sparse enough to empty a column killed the fit. A column that is empty in *every* row is still refused, by the shared check that reports it properly.

Reading coefficients after a transform When the transform is applied, coefficients are on the normal-score scale of the transformed columns, not in the original units. `summary()` says so on its `Copula` line, and the Interpretation Guide covers what the numbers then mean. Two consequences are easy to trip over:

- A coefficient is no longer "change in y per unit of x" in x's own units. If the units are the point, for instance reporting an effect per mg/dL, fit with `copula=False` and accept whatever the marginal misfit costs.
- Log-likelihoods, and so AIC and BIC, are on the transformed scale. Do not compare them across `copula` settings.

Columns left untransformed keep their own scale, so a mixed design gives a mixed coefficient vector. `copula_used_` tells you whether the transform ran at all, and the discrete-column rule above tells you which columns it reached.

If you have results from before August 2026 The transform mishandled repeated values. Its interpolation table held the raw sorted observations, which repeat whenever a column is

discrete, and every member of a block of tied values took the block's *largest* normal score instead of its mean. On a binary column that mapped 0 to +1.18 and 1 to +3.12, a column with mean 1.41 and standard deviation 0.62 where the transform is defined to deliver N(0, 1). Any repeated value was distorted in proportion to how much of the column its block spanned, and because the score a block landed on moved with the block boundaries, results also shifted between cross-validation folds: on one air quality fit every model family lost roughly half its R-squared stability with the transform engaged. Ties now take the mean of the scores their block spans, the mid-rank convention.

Separately, the skewness and excess kurtosis that drive `'auto'` were computed from raw central moments. Both are dimensionless, but that arithmetic is not: near 1e120 the cube overflows while the square is still finite, and below about 1e-150 both underflow. Either way the result was NaN, which the threshold test read as "not skewed", so a visibly skewed column silently stopped asking for the transform once its units were large enough. A column with skewness 2.52 was reported as 0 at a scale of 1e160.

Both are fixed. If you have fitted models or saved numbers from before these changes and any of your columns are discrete, integer-valued, or coarsely rounded, refit rather than reconciling.

---

5. Multiple imputation

```
MissImputer(m=20, include_y=False, max_iter=200, tol=1e-06, reg=1e-06,
            posterior=False, random_state=None)
```

FIML and multiple imputation are not rivals here. `MissImputer` estimates the joint MVN by the same full-information likelihood, then draws *m* completed data sets from it, so that NaN-intolerant downstream learners can be used without abandoning a principled missing-data model.

```
from MissLearn import MissImputer
imp = MissImputer(m=20, posterior=True, random_state=0)
results = imp.fit_transform_combine(X, y, estimator=SomeSklearnModel())
```

Methods: `fit`, `transform`, `transform_mean`, `combine`, `fit_transform_combine`, `summary`.

Tips on practical use.

- `posterior=True` adds parameter uncertainty to the draws, not just conditional variance. Without it the imputations are too similar to each other and pooled standard errors are too small.
- `include_y=True` includes the response in the imputation model. Correct when imputing for analysis; wrong if the imputed data will be used to predict that same response, since it leaks the target.
- `transform_mean` returns conditional means rather than draws. Convenient, but it is single imputation and carries the usual understatement of uncertainty.

---

6. Model selection and evaluation

6.1 NaN-safe cross-validation

```
MissKFold(n_splits=5, shuffle=False, random_state=None)
MissStratifiedKFold(n_splits=5, shuffle=True, random_state=None)
miss_cross_val_score(estimator, X, y, cv=5, scoring=None, ...)
miss_cross_validate(estimator, X, y, cv=5, scoring=None, groups=None,
                    stratified=False, ...)
```

scikit-learn's own utilities work. `cross_val_score`, `GridSearchCV` and `Pipeline` all accept MissLearn estimators with `NaN` in `X`.

The NaN-safe splitters exist for one specific case: **missing values in `y`**. MissLearn's likelihood models use rows with an unobserved response, because those rows still inform the feature distribution. scikit-learn's `StratifiedKFold` rejects them outright with `Input y contains NaN`. `MissStratifiedKFold` stratifies the observed labels normally and distributes the unlabelled rows across folds proportionally.

```
from MissLearn import miss_cross_val_score, MissStratifiedKFold
scores = miss_cross_val_score(clf, X, y_with_nan,
                              cv=MissStratifiedKFold(5))
```

If `y` is complete, use whichever you prefer.

Warm starting does not apply across folds. `miss_cross_val_score` and `miss_cross_validate` strip the stored parameter vector from every per-fold estimator copy. Successive training sets overlap by (k-1)/k, so a fold started from the previous fold's optimum would be initialised from parameters fitted partly on its own held-out rows. The cost is optimizer iterations; the alternative is a fold that has seen its own test data.

6.2 Hyperparameter search

`GridSearchCV` works directly:

```
from sklearn.model_selection import GridSearchCV
from MissLearn import MissRidgeRegressor

search = GridSearchCV(MissRidgeRegressor(compute_se=False),
                      {'alpha': [0.01, 0.1, 1.0, 10.0]}, cv=5).fit(X, y)
```

Set `compute_se=False` during search. Standard errors are computed per fit and are wasted work when you are only ranking configurations.

6.3 Scoring

`score` gives R^2 for regressors and accuracy for classifiers, matching scikit-learn. On imbalanced problems prefer ROC-AUC and the Brier score; accuracy hides a great deal. The Brier score in particular is where misspecified generative models reveal themselves.

7. Diagnosis and model choice

7.1 MissDiagnostic

```
MissDiagnostic(X, y=None, feature_names=None, alpha=0.05)
```

Characterises the missingness before you model it. Methods: `little_mcar_test`, `mar_plausibility`, `pattern_summary`, `missingness_correlations`, `descriptive_by_missingness`, `summary`.

```
from MissLearn import MissDiagnostic
d = MissDiagnostic(X, y, feature_names=names).fit()
d.summary()
```

Read it as a decision procedure. If MCAR is not rejected, deletion is unbiased though wasteful. If MCAR is rejected but missingness is predictable from observed columns, MAR is defensible and FIML is well founded. If MCAR is rejected and nothing observed explains the pattern, MNAR cannot be excluded and a sensitivity analysis is mandatory rather than optional.

7.2 MissRecommender

```
MissRecommender(task='auto', groups=None, feature_names=None, alpha=0.05,
                drop_threshold=0.6, high_missing=0.3, gp_max_n=1000,
                probe_nonlinearity=True, probe_max_n=2000, random_state=0)

recommend_model(X, y, **kwargs)     # convenience wrapper
```

Turns the diagnosis into a ranked list of families **with the reasoning attached**. It gathers the mechanism, the shape of the problem, tail behaviour, grouping structure and a cheap linearity probe, then scores the families and reports why.

```
rec = recommend_model(X, y, feature_names=names)
print(rec.recommended_)
for entry in rec.ranked_:
    print(entry)
rec.summary()
```

Attributes: `recommended_`, `ranked_`, `evidence_`, `preprocessing_`, `followups_`, `notes_`.

It also reports columns to drop rather than model (missing rate above `drop_threshold`), sets `copula=True` when tails demand it, separates outright vetoes from low scores, and marks `MissSensitivity` as required when MNAR cannot be excluded.

Read `evidence_` before `recommended_`. The evidence is the part you can check against what you know about how the data were collected.

Its limits are real. The linearity probe compares a linear model against a nearest-neighbour model. When those are close it prefers the simpler model, and on data where an RBF kernel would win but nearest neighbours does not, it recommends the wrong family. The shipped thyroid example is exactly that case. The recommender ranks families worth trying; the trying still has to happen.

8. Inspection and interpretation

8.1 Coefficients and inference

The likelihood families expose `coef_`, `coef_std_`, `conf_int(alpha)`, `loglik_`, `aic_`, `bic_` and `converged_`, and a `summary()` that formats coefficients with standard errors, z statistics and p-values.

Check `converged_` before reading any of it.

8.2 SHAP explanations

```
MissExplainer(model, exact_threshold=15, n_kernel_samples=512,
              output='auto', class_index=None, random_state=None)
```

`MissExplainer` is the interface most users want. `MissShapley`, also exported, is the engine underneath it: the exact enumeration and the kernel sampler, without the plotting and the missingness attribution layered on top. Reach for it only if you are building something other than `MissExplainer` on the same value function.

A FIML model computes E[Y | X_observed] exactly for *any* subset of observed features, so the model is its own exact coalition value function. Removing a feature from a coalition means setting it to `NaN` and evaluating there. No background data set, no sampling of replacement values.

Two distinct explanations:

Value SHAP, `shap_values(X)`. How much each observed value moved the prediction relative to the all-unknown baseline `expected_value_`. Unobserved features get exactly zero credit, since whatever the model inferred about them came from their observed neighbours.

Missingness SHAP, `miss_shap(X)`. How much *observing* each feature would be worth. This answers "which measurement should we pay for next?", which value SHAP cannot.

The `output` parameter matters on classifiers. `predict` returns a hard label, so using it as a coalition value collapses every Shapley difference to zero on imbalanced data. `'auto'` uses predicted probability for a binary classifier and the prediction itself for a regressor. `'proba'` and `'log-odds'` name those scales explicitly; `'raw'` is the old label behaviour and exists only for comparison. `value_scale_` reports what was used. A multi-class model has no single scalar value function and requires `class_index`.

Plotting helpers take the computed values first, then the data: `plot_beeswarm(phi, X)`, `plot_waterfall(phi, X, i=0)`, `plot_miss_importance(psi)`, `plot_dependence(phi, X, feature_idx)`.

8.3 Sensitivity analysis

```
MissSensitivity(estimator, delta_range=(-3.0, 3.0), n_delta=25, m=10,
                standardise=True, alpha=0.05, imputer_reg=1e-06,
                random_state=None)
```

FIML assumes MAR. This asks what would happen if it were wrong: if the missing responses were systematically higher or lower than the model believes, by delta standard deviations, at what point does your conclusion change?

```
sens = MissSensitivity(MissLinear()).fit(X, y, feature_names=names)
sens.summary()
print(sens.verdict(coef_idx=2))
```

`verdict()` returns one of three labels, not four. `ROBUST` covers both the case that never tips within the grid and a tipping point at or beyond 1.0 sigma. `MILD` is 0.5 to 1.0 sigma. `SENSITIVE` is below 0.5 sigma.

A `SENSITIVE` verdict does not mean the finding is wrong. It means the finding rests on the MAR assumption, and you must argue for MAR on substantive grounds or soften the claim.

The estimator must expose `coef_` after fitting. Given one that does not, such as `MissBayesRegressor`, `MissSupport*`, `MissNeighbors*` or `MissGaussian*`, `fit` raises `AttributeError` naming the alternatives. It previously substituted zeros, which made every conclusion look maximally robust, and that is the most dangerous way for this particular tool to fail.

---

9. Computational considerations

9.1 Complexity

The per-pattern likelihood costs $O(p^3)$ for the Cholesky factorisation and triangular solves. With pattern grouping the total is **$O(G \cdot p^3)$** per iteration, where G is the number of distinct missingness patterns.

The practical consequence is that the *structure* of the missingness matters more than its rate. Blockwise missingness, where whole groups of columns go missing together, keeps G small. Scattered missingness at the same rate can produce nearly one pattern per row.

9.2 Practical limits

Family	Comfortable scale	Limiting factor
Linear, penalized, mixed	large n, p up to ~50	$O(G \cdot p^3)$ per iteration
Generative	large n and p	closed form, no optimisation
Neighbours	moderate n	pairwise distances
Support vector	moderate n	kernel matrix
Gaussian process	$n \lesssim 1000$	exact inference is $O(n^3)$

Use `compute_se=False` to skip the numerical Hessian, and `warm_start=True` when refitting the same estimator repeatedly outside cross-validation. `clear_cache()` empties the module-level quadrature caches.

9.3 How the classifiers marginalise, and what `n_quadrature` does

Every classifier with a logistic link has to average the link over the missing features. For a row with observed part `x_obs`, that average is

```
P(y = 1 | x_obs) = E[ sigma(a + t) ],   t ~ N(0, v)
```

where `a` collects the intercept and the observed contribution, and `v` is the variance the missing features contribute. **`v` grows with both how many features are missing and how large the coefficients are**, which is the fact that matters below: across the missingness patterns of a fitted model with coefficients up to 3.6, the median `v` was 23, and at coefficients up to 9.9 it was 121.

Gauss-Hermite quadrature, the obvious rule, fails here for a structural reason rather than for want of nodes. In its own variable the integrand is a step of width about `1/sqrt(2v)`, so as `v` grows the rule is asked to resolve a sharper and sharper edge with nodes that do not move. Raising `n_quadrature` from 20 to 640 still left an error above 1e-4 at `v = 100`. At the default of 20 nodes the worst error in a returned probability was 0.004 by `v = 25` and 0.03 by `v = 100`.

Wide variances are now handled by splitting the logistic into a unit step plus a remainder. The step integrates exactly, and the remainder is bounded by `exp(-|s|)`, so its support does not widen with `v` however large `v` becomes. The result is accurate to about 1e-13 for every variance from 0.1 to 2000. Gauss-Hermite is kept below `v = 1`, where it is already exact to 1e-11.

What this means for `n_quadrature`. It sizes the small-variance branch only, and it no longer changes your answers. Before, fitting and prediction used different rules and the fit was biased where the signal was strongest: raising the node count from 20 to 320 moved a coefficient by 0.16. Now both use the same rule, and the default 20-node fit lands on what 320 nodes were previously needed to reach. There is no longer a reason to raise it, and nothing to gain from tuning it.

`MissMixedClassifier` is different. Its integral is over the random effect, and its integrand is a product over a subject's observations rather than a single logistic, so the split does not apply: there is no step to peel off a product. It uses adaptive Gauss-Hermite instead, placing each subject's nodes on that subject's own posterior rather than on the shared prior. This matters more than it sounds: a subject with several observations has its effect pinned into a narrow interval that may sit far from zero, and unadapted nodes then sample where the integrand is negligible. The error grew with the random-effect scale *and* with observations per subject, reaching 1.8 in a log-likelihood at `tau = 5` with twenty observations per subject, where even 320 unadapted nodes left 6e-4. Adapted, the same case is accurate to 8e-6 at the default 20 nodes. Ordinary random effects, `tau` below about 1.5, were never affected either way.

Cost follows the same shape as the accuracy. Rows whose variance is small take the same path they always did, marginally faster for skipping the check; rows in the regime that used to be wrong cost two to six times more per objective evaluation. If a fit that used to finish is now slower, this is why, and the answer it used to give was wrong.

---

10. Common pitfalls and recommended practices

Sentinel values are not missing values. A recorded serum insulin of exactly 0 is physiologically impossible and encodes a measurement never taken. Convert sentinels to `NaN` before modelling. No amount of careful modelling afterwards recovers from treating them as real.

Compare like with like. A difference between arms is evidence about the missing-data treatment only if nothing else differs. Hold the model class fixed, give conventional arms the same standardisation the MissLearn estimator applies internally, tune regularisation on both sides

over the same grid, and give a Gaussian-process comparator the same learnable noise term. Each of those controls has changed a conclusion in this project's own benchmarks.

Grouped data needs grouped folds. If rows cluster by patient, site or batch, a shuffled split leaks identity and answers a different question.

Read the Brier score, not just accuracy. A misspecified generative model can look reasonable on AUC while being badly calibrated.

A negative result is a question. When a MissLearn model underperforms, ask which assumption is binding: the Gaussian working model for the features, or the response model. They are separate, and the answer is often the second.

Check `converged_`. An unconverged fit's coefficients and standard errors are not worth interpreting.

MNAR cannot be tested away. If the diagnosis leaves MNAR open, run `MissSensitivity` and report what it says.

---

11. Interoperating with scikit-learn

MissLearn estimators pass the ordinary scikit-learn contract:

```
from sklearn.base import clone
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import GridSearchCV, cross_val_score

clone(MissLinear())                                  # works
Pipeline([('s', StandardScaler()), ('m', MissLinear())]).fit(X, y)
GridSearchCV(MissRidgeRegressor(), {'alpha': [0.1, 1]}, cv=3).fit(X, y)
cross_val_score(MissLinear(), X, y, cv=3)            # NaN in X is fine
```

`MissLinear` resolves as `MissLinear -> MissBase -> BaseEstimator -> RegressorMixin`, reports `_estimator_type` as `regressor`, and sets `n_features_in_` and `feature_names_in_` at fit time. Metadata routing is supported through `set_fit_request` and `set_predict_request`.

pandas is handled transparently: pass a DataFrame and column names propagate into `feature_names_in_`, so summaries and reports name the measurement rather than a column index.

Two things to know:

- The estimator tag `allow_nan` currently reports `False`, even though the estimators do accept `NaN`. The common paths above are unaffected, but a third-party utility that consults the tag may refuse input it could have handled.
- scikit-learn's `StratifiedKFold` rejects `NaN` in `y`. Use `MissStratifiedKFold` when the response is incomplete (section 6.1).

11.1 Checking your own estimator

```
from MissLearn import check_missing_data_estimator
```

```
report = check_missing_data_estimator(estimator, task=None, seed=0,
                                      fit_kwargs=None,
                                      determinism='default')
print(report)
```

`check_estimator` verifies the scikit-learn contract but says nothing about incomplete data, because scikit-learn estimators mostly refuse it. This drives any NaN-tolerant estimator through eleven degenerate regimes and reports what it does in each: no complete cases, an entirely absent column, a single observed cell, an absent target, blockwise missingness, a design wider than it is tall, one feature, extreme scales.

Nothing in it depends on MissLearn, so it works on your own estimator or a third party's.

The distinction it draws is the useful part. **A clear refusal is acceptable**, because an exception stops a pipeline and tells the caller what happened. **A silent `NaN` never is**, because it propagates into whatever consumes the prediction and is indistinguishable from an answer. A refusal whose message does not say why is reported separately, since refusing without naming the cause leaves the user unable to act.

```
class MyEstimator(RegressorMixin, BaseEstimator):
    def fit(self, X, y=None, **kw):
        self.n_features_in_ = X.shape[1]
        return self
    def predict(self, X):
        return np.full(X.shape[0], np.nan)      # the forbidden outcome

report = check_missing_data_estimator(MyEstimator())
report.ok        # False
print(report)    # names the regimes that need attention, and why
```

The returned `MissingDataReport` renders itself when printed, so echoing it in a notebook gives the report rather than an address.

---

12. API reference

Estimators. Each family provides a regressor, a classifier, and a dispatcher that selects between them from `y`.

Family	Regressor	Classifier	Dispatcher
Linear	`MissLinear`	`MissLogistic`	
Ridge	`MissRidgeRegressor`	`MissRidgeClassifier`	`MissRidge`
LASSO	`MissLASSORegressor`	`MissLASSOClassifier`	`MissLASSO`
Generative	`MissBayesRegressor`	`MissBayesClassifier`	`MissBayes`
Neighbours	`MissNeighborsRegressor`	`MissNeighborsClassifier`	`MissNeighbors`
Support vector	`MissSupportRegressor`	`MissSupportClassifier`	`MissSupport`
Gaussian process	`MissGaussianRegressor`	`MissGaussianClassifier`	`MissGaussian`
Mixed effects	`MissMixedRegressor`	`MissMixedClassifier`	`MissMixed`

Meta-estimators. `MissEnsemble`, `MissMulticlass`, `MissPreprocessor`.

Tools. `MissImputer`, `MissDiagnostic`, `MissRecommender`, `MissExplainer`, `MissSensitivity`.

Cross-validation. `MissKFold`, `MissStratifiedKFold`.

Functions. `prefit_check`, `recommend_model`, `miss_cross_val_score`, `miss_cross_validate`, `clear_cache`.

Attributes common to every fitted estimator

`n_features_in_`, `feature_names_in_`, `n_samples_fit_`, `n_missing_X_`, `n_missing_y_`, `missing_rate_X_`, `n_complete_`, `n_partial_`, `copula_used_`.

`missingness_report()` prints these, and `get_feature_names_out()` follows the scikit-learn transformer convention.

Additional attributes by family

Family	Adds
Likelihood models	`coef_`, `intercept_`, `coef_std_`, `loglik_`, `aic_`, `bic_`, `converged_`, `mu_X_`/`mu_joint_`, `Sigma_X_`/`Sigma_joint_`
All classifiers	`classes_`
Generative	`class_prior_`, `cov_k_`, `mu_jk_`, `intercepts_`, `mu_Y_`
Neighbours	`X_train_`, `Z_train_`, `mu_`, `n_neighbors_`
Support vector	`n_support_`, `class_prior_`
Gaussian process	`length_scale_`, `log_marginal_likelihood_`
Mixed effects	`blup_`, `tau_sq_`, `sigma_sq_`, `icc_`, `groups_fit_`

MissLearn 0.9.2. Signatures and defaults in this guide are transcribed from the installed package. If one disagrees with the code, the code is right; please report it.